

MAROOM Backend API Guide

Frontend Integration Reference

Generated from the Spring backend controllers and DTOs in
`backend/src/main/java/com/maroom/maroom`.

Document date: 2026-05-10

Default local base URL: `http://localhost:8080`

1. Integration Basics

- Base URL in the frontend client is `process.env.EXPO_PUBLIC_API_URL`, falling back to `http://localhost:8080`.
- Use Content-Type: `application/json` for request bodies.
- Login and signup return a bearer token. The current frontend interceptor sends `Authorization: Bearer <token>` automatically.
- Most domain endpoints still identify the active user by `userId` in the path/body. Token enforcement is not yet centralized.
- UUID fields are strings. Timestamps are ISO-8601 strings.
- Error bodies are not normalized yet: some endpoints return strings, some return maps such as `{"error":"..."}`, and uncaught exceptions may use Spring's default error payload.

2. Endpoint Overview

Area	Method	Endpoint	Frontend use
Health	GET	/hello	Backend availability check.
Auth	POST	/auth/signup	Create account and receive token + <code>userId</code> .
Auth	POST	/auth/login	Login and receive token + <code>userId</code> + onboarding flag.
Auth	POST	/auth/logout	Delete current session token. Requires Authorization header.
Preferences	GET	/preferences/options	Get allowed onboarding option values.
Preferences	POST	/preferences	Save onboarding preferences.
Preferences	GET	/preferences/{userId}	Load saved onboarding preferences.
Furniture	GET	/api/furniture-items	List all furniture.
Furniture	GET	/api/furniture-items/search	Explore search and filters.
Furniture	GET	/api/furniture-items/{id}	Product detail page.
Furniture	POST	/api/furniture-items	Internal/admin create item.
Swipe	GET	/swipe/feed/{userId}?size=20	Recommended feed. Falls back to unswiped random items.
Swipe	POST	/swipe	Record left/right swipe; RIGHT also saves to Liked.
Swipe	GET	/swipe/events/{userId}	Debug/history view of user swipes.
Saved	GET	/saved/{userId}	Saved products for the default Liked list.
Saved	DELETE	/saved/{userId}/{furnitureId}	Remove a furniture item from Liked.
Saved	GET	/saved/lists/{userId}	Fetch all custom saved lists.
Saved	POST	/saved/lists	Create a saved list.
Saved	GET	/saved/items/{listId}	Fetch raw saved items by list.
Saved	POST	/saved/items	Add raw saved item to a list.
Internal	POST	/api/furniture/import/large	Bulk import furniture data.
Internal	POST	/api/furniture-items/embeddings/backfill	Backfill embedding vectors.
Internal	POST	/swipe/user-embedding/{userId}/recompute	Recompute user embedding.
Users	Mixed	/users, /users/{id}	Basic user CRUD-like helpers; treat as internal unless needed.

3. Shared Shapes

FurnitureItem

```
{
  "id": "uuid",
  "title": "Article Sven Sofa",
  "category": "Sofa",
  "brand": "Article",
  "style": "MODERN",
  "color": "Warm beige",
  "price": 1299,
  "roomType": "Living Room",
  "productUrl": "https://...",
  "imageUrl": "https://..."
}
```

AuthResult

```
{
  "token": "base64url-session-token",
  "userId": "uuid",
  "hasCompletedOnboarding": false
}
```

Preference

```
{
  "userId": "uuid",
  "homeType": "1_BEDROOM",
  "roomSize": "MEDIUM",
  "styles": "[\"MODERN\", \"JAPANDI\"]",
  "colorPalette": "[\"WARM_NEUTRAL\"]",
  "minBudget": 100,
  "maxBudget": 500
}
```

Current backend response stores styles and colorPalette as JSON strings, not JSON arrays. Frontend should parse them if it needs array semantics.

4. Auth APIs

POST /auth/signup

Method	Path	Purpose
POST	/auth/signup	Create a local email user and start a session.

Status: 200 OK on success. Duplicate email currently throws an uncaught `IllegalArgumentException`.

Request

```
{
  "email": "new.user@example.com",
  "password": "plain-password",
  "displayName": "New User"
}
```

Success response

```
{
  "token": "P_sX...",
  "userId": "9fdc9d49-7b93-4ac7-b4e5-a9ad6f45d101",
  "hasCompletedOnboarding": false
}
```

Frontend notes

- Store token and userId after success.
- Signup uses authProvider EMAIL internally.

POST /auth/login

Method	Path	Purpose
POST	/auth/login	Login with email/password.

Status: 200 OK on success. Invalid credentials currently throw an uncaught `IllegalArgumentException`.

Request

```
{
  "email": "new.user@example.com",
  "password": "plain-password"
}
```

Success response

```
{
  "token": "P_sX...",
  "userId": "9fdc9d49-7b93-4ac7-b4e5-a9ad6f45d101",
  "hasCompletedOnboarding": true
}
```

Frontend notes

- hasCompletedOnboarding is true when a Preference row exists for the user.

POST /auth/logout

Method	Path	Purpose
POST	/auth/logout	Delete the active session token.

Status: 200 OK on success. 400 "Invalid token" if the header is missing or malformed.

Request

```
Authorization: Bearer <token>
```

Success response

```
Logged out successfully
```

Frontend notes

- Response is plain text, not JSON.
- No request body is required.

5. Preferences / Onboarding APIs

GET /preferences/options

Method	Path	Purpose
GET	/preferences/options	Return enum-like onboarding option values.

Status: 200 OK

Success response

```
{
  "homeTypes": ["SINGLE_FAMILY_HOME", "1_BEDROOM", "2_BEDROOM", "DORM_STUDIO"],
  "roomSizes": ["SMALL", "MEDIUM", "LARGE"],
  "styles": ["MINIMALIST", "MODERN", "SCANDINAVIAN", "MID_CENTURY", "JAPANDI", "BOHO", "INDUSTRIAL"],
  "colorPalettes": ["WARM_NEUTRAL", "COOL_NEUTRAL", "EARTHY_TONES", "BLACK_WHITE", "VIBRANT", "PASTEL"],
  "budgetTiers": ["UNDER_100", "100_300", "300_500", "500_PLUS"]
}
```

POST /preferences

Method	Path	Purpose
POST	/preferences	Create or update a user's onboarding preferences.

Status: 200 OK. 400 with a plain text validation message on invalid input.

Request

```
{
  "userId": "9fdc9d49-7b93-4ac7-b4e5-a9ad6f45d101",
  "homeType": "1_BEDROOM",
  "roomSize": "MEDIUM",
  "styles": ["MODERN", "JAPANDI"],
  "colorPalette": ["WARM_NEUTRAL"],
  "minBudget": 100,
  "maxBudget": 500
}
```

Success response

```
{
  "userId": "9fdc9d49-7b93-4ac7-b4e5-a9ad6f45d101",
  "homeType": "1_BEDROOM",
  "roomSize": "MEDIUM",
  "styles": ["MODERN", "JAPANDI"],
  "colorPalette": ["WARM_NEUTRAL"],
  "minBudget": 100,
  "maxBudget": 500
}
```

Frontend notes

- styles must contain 1 to 3 allowed values.
- colorPalette must contain 1 to 2 allowed values.
- minBudget and maxBudget are required and cannot be negative.

GET /preferences/{userId}

Method	Path	Purpose
GET	/preferences/{userId}	Load one user's saved onboarding preferences.

Status: 200 OK or 404 Not Found.

Success response

```
Preference object
```

6. Furniture APIs

GET /api/furniture-items

Method	Path	Purpose
GET	/api/furniture-items	Return all furniture items.

Status: 200 OK

Success response

```
[ FurnitureItem, ... ]
```

GET /api/furniture-items/search

Method	Path	Purpose
GET	/api/furniture-items/search	Search and filter furniture items.

Status: 200 OK

Request

```
Query parameters:
q=sofa
brand=Article
category=Sofa&category=Chair
roomType=Living%20Room
color=warm%20beige
minPrice=100
maxPrice=1000
sortBy=price-low-to-high | price-high-to-low
```

Success response

```
[ FurnitureItem, ... ]
```

Frontend notes

- category, roomType, and color can be repeated query parameters.
- q matches title only.
- category also checks roomType for matching category-like values.
- color token matching ignores punctuation and only uses tokens with length >= 3.

GET /api/furniture-items/{id}

Method	Path	Purpose
GET	/api/furniture-items/{id}	Return a furniture item by UUID.

Status: 200 OK or 404 Not Found.

Success response

```
FurnitureItem
```

POST /api/furniture-items

Method	Path	Purpose
POST	/api/furniture-items	Create a furniture item. Intended for admin/internal use.

Status: 200 OK

Request

```
{
  "title": "Article Sven Sofa",
  "category": "Sofa",
  "brand": "Article",
  "style": "MODERN",
  "color": "Warm beige",
  "price": 1299,
  "roomType": "Living Room",
  "productUrl": "https://...",
  "imageUrl": "https://..."
}
```

Success response

```
FurnitureItem with generated id
```

Frontend notes

- Backend attempts to create an embedding after save when embedding config is available.

7. Swipe / Recommendation APIs

GET /swipe/feed/{userId}?size=20

Method	Path	Purpose
GET	/swipe/feed/{userId}?size=20	Return recommendation feed for a user.

Status: 200 OK

Success response

```
[ FurnitureItem, ... ]
```

Frontend notes

- size <= 0 returns an empty list.
- size is capped at 100.
- If embeddings are unavailable or insufficient, backend fills with unswiped random furniture.

POST /swipe

Method	Path	Purpose
POST	/swipe	Record a swipe event for a user/furniture pair.

Status: 200 OK, 400 for missing fields, 409 if this user already swiped this furniture.

Request

```
{
  "userId": "9fdc9d49-7b93-4ac7-b4e5-a9ad6f45d101",
  "furnitureId": "38ef9df6-94cc-4bc4-97a3-52edb1c9d2ee",
  "direction": "RIGHT"
}
```

Success response

```
{
  "ok": true,
  "swipeEventId": "uuid",
  "savedToLiked": true,
  "userEmbedding": {
    "storageReady": true,
    "updated": true,
    "userId": "uuid",
    "positiveCount": 4,
    "negativeCount": 2,
    "swipedEmbeddingCount": 6,
    "skippedMissingFurnitureEmbeddings": 0
  }
}
```

Frontend notes

- direction must be LEFT or RIGHT.
- RIGHT automatically creates/uses the Liked saved list and saves the item once.
- The response includes embedding recompute status; frontend can usually ignore it.

GET /swipe/events/{userId}

Method	Path	Purpose
GET	/swipe/events/{userId}	Return swipe history for a user, newest first.

Status: 200 OK

Success response

```
[
  {
    "id": "uuid",
    "userId": "uuid",
    "furnitureId": "uuid",
    "direction": "RIGHT",
    "createdAt": "2026-05-10T12:34:56Z"
  }
]
```

8. Saved APIs

GET /saved/{userId}

Method	Path	Purpose
GET	/saved/{userId}	Return saved furniture in the user's Liked list.

Status: 200 OK. Empty array when no Liked list exists.

Success response

```
[
  {
    "savedItemId": "uuid",
    "savedAt": "2026-05-10T12:34:56Z",
    "furnitureId": "uuid",
    "title": "Article Sven Sofa",
    "category": "Sofa",
    "brand": "Article",
    "style": "MODERN",
    "color": "Warm beige",
    "price": 1299,
    "roomType": "Living Room",
    "productUrl": "https://...",
    "imageUrl": "https://..."
  }
]
```

Frontend notes

- Frontend should map furnitureId to Product.id if using the Product shape.

DELETE /saved/{userId}/{furnitureId}

Method	Path	Purpose
DELETE	/saved/{userId}/{furnitureId}	Remove a furniture item from the user's Liked list.

Status: 200 OK, 404 {"ok":false,"message":"Liked list not found"} or "Saved item not found".

Success response

```
{"ok": true}
```

GET /saved/lists/{userId}

Method	Path	Purpose
GET	/saved/lists/{userId}	Return all saved lists for a user.

Status: 200 OK

Success response

```
[
  {
    "id": "uuid",
    "userId": "uuid",
    "name": "Liked",
    "createdAt": "2026-05-10T12:34:56Z"
  }
]
```

POST /saved/lists

Method	Path	Purpose
POST	/saved/lists	Create a saved list.

Status: 200 OK

Request

```
{
  "userId": "9fdc9d49-7b93-4ac7-b4e5-a9ad6f45d101",
  "name": "Liked"
}
```

Success response

```
SavedList
```

GET /saved/items/{listId}

Method	Path	Purpose
GET	/saved/items/{listId}	Return raw saved item records for a list, newest first.

Status: 200 OK

Success response

```
[
  {
    "id": "uuid",
    "savedListId": "uuid",
    "furnitureId": "uuid",
    "createdAt": "2026-05-10T12:34:56Z"
  }
]
```

POST /saved/items

Method	Path	Purpose
POST	/saved/items	Add a raw saved item record to a list.

Status: 200 OK

Request

```
{
  "savedListId": "uuid",
  "furnitureId": "uuid"
}
```

Success response

```
SavedItem
```

Frontend notes

- This direct endpoint does not de-dupe. The swipe RIGHT path does de-dupe against the Liked list.

9. User Helper APIs

POST /users

Method	Path	Purpose
POST	/users	Create a user without auth session handling. Treat as internal unless the frontend needs it.

Status: 201 Created, 400 {"error": "Email is required"}, 409 {"error": "Email already exists"}.

Request

```
{
  "email": "new.user@example.com",
  "displayName": "New User",
  "authProvider": "LOCAL"
}
```

Success response

```
User
```

Frontend notes

- Use /auth/signup for normal email signup because it hashes password and returns a session token.

GET /users

Method	Path	Purpose
GET	/users	List all users. Internal/debug use.

Status: 200 OK

Success response

```
[ User, ... ]
```

Frontend notes

- Current User JSON includes passwordHash. Do not use this in normal frontend screens.

GET /users/{id}

Method	Path	Purpose
GET	/users/{id}	Fetch a user by UUID. Internal/debug use.

Status: 200 OK or 404 {"error": "User not found"}.

Success response

```
User
```

10. Internal / Operations APIs

POST /api/furniture/import/large

Method	Path	Purpose
POST	/api/furniture/import/large	Import the large furniture CSV dataset.

Status: 200 OK

Success response

```
Imported 123 furniture items.
```

Frontend notes

- Plain text response.
- Use manually/admin-side; not needed by the mobile app.

POST /api/furniture-items/embeddings/backfill?limit=25&batchSize=25

Method	Path	Purpose
POST	/api/furniture-items/embeddings/backfill?limit=25&batchSize=25	Backfill missing furniture embeddings.

Status: 200 OK

Success response

```
{
  "configured": true,
  "storageReady": true,
  "limit": 25,
  "batchSize": 25,
  "checked": 25,
  "embedded": 20,
  "skipped": 5,
  "failed": 0,
  "batches": 1
}
```

Frontend notes

- limit is clamped to 1..500.
- batchSize is clamped to 1..50.
- Requires embedding configuration.

POST /swipe/user-embedding/{userId}/recompute

Method	Path	Purpose
POST	/swipe/user-embedding/{userId}/recompute	Recompute the user's preference embedding from swipe history.

Status: 200 OK

Success response

```
{
  "storageReady": true,
  "updated": true,
  "userId": "uuid",
  "positiveCount": 4,
  "negativeCount": 2,
  "swipedEmbeddingCount": 6,
  "skippedMissingFurnitureEmbeddings": 0
}
```

Frontend notes

- Usually triggered automatically by POST /swipe.

GET /hello

Method	Path	Purpose
GET	/hello	Simple backend health check.

Status: 200 OK

Success response

```
maroom backend running
```

11. Suggested Frontend Flow

- Signup/login via /auth/signup or /auth/login, then persist token and userId.
- If hasCompletedOnboarding is false, call GET /preferences/options and POST /preferences.
- Home feed can call GET /swipe/feed/{userId}?size=20.
- On each card action, call POST /swipe. A RIGHT swipe also saves the item to Liked.
- Explore filters should call GET /api/furniture-items/search with repeated query params for multi-select filters.
- Saved page should call GET /saved/{userId}; unsave via DELETE /saved/{userId}/{furnitureId}.